

OmniStat

OmniStat is a polyglot, self-hosted telemetry and observability engine designed to provide total, unblinking visibility into your development lifecycle.



ARCHITECTURE

POLYGLOT

STATUS

ACTIVE DEVELOPMENT

Far beyond standard profile stats, OmniStat acts as a background intelligence pipeline. It continuously mines both public and private repository activity, reduces massive datasets into behavioral metrics, and projects that data onto a highly stylized, strict-terminal HUD.

The Philosophy

Standard metrics tell you *what* you built. OmniStat tells you *how* you build. By tracking temporal heatmaps, absolute byte-level code churn, and repository velocity, OmniStat acts as a personal, enterprise-grade auditing tool for your engineering habits.

The Trinity Architecture

OmniStat operates across three distinct, highly optimized layers:

1. The Forge (Data Ingestion)

Technology: `Scala` + `sttp` + `circe` + `PostgreSQL` A headless background worker that wakes up on a `cron`-schedule. It interfaces with the **GitHub GraphQL API** to extract deeply nested telemetry data, applies functional transformations to aggregate byte counts and temporal patterns, and safely upserts the intelligence into a PostgreSQL database.

2. ⚡ The Gateway (API Proxy)

Technology: `Go` (`net/http`) Built with raw Go, this layer acts as the blisteringly fast **Backend-For-Frontend (BFF)**. It handles database reads for instant metric delivery and serves as a secure reverse proxy, shielding the heavy Scala engine from the public internet.

3. The Terminal (Visualization)

Technology: `Next.js` + `TailwindCSS` + `SWR` The user-facing HUD. Built strictly with a **neo-brutalist, ctOS-inspired aesthetic** (deep black backgrounds, phosphor green accents, and monospace typography). It consumes the Go gateway's data to power live-streaming terminal feeds, glowing language radars, and punch-card velocity matrices.

⚡ Key Capabilities

- **Absolute Language Tracking:** Analyzes exact byte distribution across the entire codebase portfolio, exposing true stack proficiency.

- ☉ **Temporal Heatmapping:** Identifies "Golden Hours" by correlating commit density with specific times of day and days of the week.
- ☒ **Live-Stream Introspection:** Visualizes recent codebase actions in a simulated, typewriter-effect terminal feed.
- ☒ **Type-Safe Boundaries:** Ensures complex data structures processed in the JVM (Scala) are perfectly mirrored in the TS environment via the Go intermediary.

☒ Setup & Deployment

[The Forge](#)

```
cd code-telemetry-engine/data
sbt run
```

[The Gateway](#)

```
cd code-telemetry-engine/backend
go run cmd/api/main.go
```

[The Terminal](#)

```
cd frontend
npm run dev
```

[!IMPORTANT] OmniStat requires a GitHub Personal Access Token with `repo` and `read:user` scopes to function. Configure this in the Forge's environment variables.

(C) 2026 BLUME_CORP // DATA_SEC_PROTOCOL

☒ Project Roadmap

- ☒ **Phase 1:** Polyglot Telemetry Core Setup.
- ☒ **Phase 2:** Data Ingestion Layer & Backend.
- ☒ **Phase 3:** Frontend UI Foundations.
- ☐ **Phase 4:** Feature Implementation & Dashboard Integration.
- ☐ **Phase 5:** Full-Stack Polish.